

Implementation of Non-Orthogonal Correctors

In this document, I will be going over the non-orthogonal correctors and its implementation into the accompanying code in the [Github repo](#).

This document will cover what the problem being addressed is, the solution (non-orthogonal correctors), and how it is implemented into the code.

I learned from and really got all the information in this document from the following lectures (which I highly recommend):

- [\[CFD\] What is Mesh Non-Orthogonality?](#)
- [\[CFD\] Mesh Non-Orthogonality 2: The Over-Relaxed Approach](#)
- [\[CFD\] Green-Gauss Cell-Based and Node-Based Gradient Schemes](#)
- [\[CFD\] Green-Gauss Node-Based Gradient Schemes](#)

Honestly, this document is more for me so that I can keep track of the implementation for debugging (which helped a lot since writing things down kind of makes you question more things which helps with understanding).

I also refer to something called the “main” document which is the *SIMPLE Algorithm using FVM for 2D Incompressible N-S Eqns* pdf.

Okay, let’s begin!

Contents

The Problem	2
Defining Non-Orthogonality.....	2
When Non-Orthogonality becomes a Problem	2
The Solution	4
General Idea	4
Green-Gauss Node-Based Gradient.....	6
The Implementation	9
Where to Implement this Correction	9
How to Implement this Correction	10
Implementation at Boundaries	13

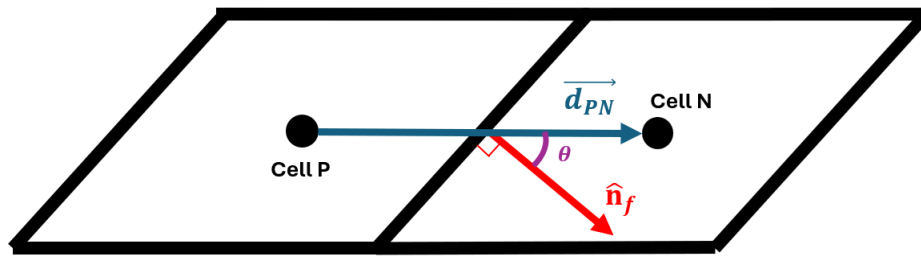
The Problem

In order to understand non-orthogonal correctors, we need to understand what problem they are actually addressing. Matter of fact, we need to understand what non-orthogonality is in the first place. As such, let's start with a definition of non-orthogonality and why it becomes problematic in CFD codes when our mesh is non-orthogonal.

Defining Non-Orthogonality

Non-orthogonality is a characteristic of a cell face in the mesh that defines the deviation of the vector between adjacent cell centers (adjacent to the face of interest) and the normal vector of the face. A key point here that people glance over is that non-orthogonality is evaluated at a face, not a cell.

This definition in words is good but a picture does it more justice so a diagram is shown below. The **non-orthogonality angle** (θ) is the angle between those **center-to-center vector** ($\vec{d}_{PN}$) and the **face normal vector** ($\hat{n}_f$).



In our meshers and modern CFD codes, there are slightly different definition of non-orthogonality such as the cell-based one covered in the [video](#). But, for our computations, the face non-orthogonality is of interest and what will be corrected.

Also note that meshers and modern CFD code normalize the non-orthogonality angle by the largest deviation angle (90 degrees) in order to get a quality value which is what users often use to evaluate their meshes.

Now that we understand what face non-orthogonality is let us consider why it is a problem.

When Non-Orthogonality becomes a Problem

To consider when this becomes a problem, let us recall the conservation of momentum equation in vector form for the steady incompressible N-S equations as in Equation (1) below.

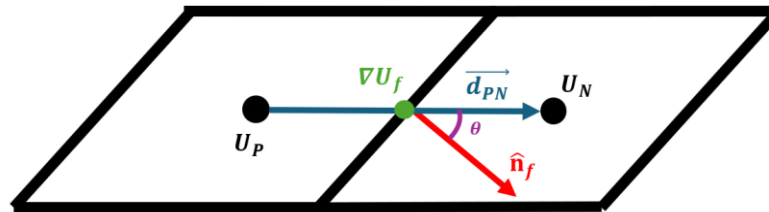
$$\nabla \cdot (UU^T) - \nabla \cdot (\nu \nabla U) = -\nabla p \quad (1)$$

For this equation, we would like to implement a finite volume approach which includes using the divergence theorem then summing the contributions from each face. I will not go into detail on that but what ends up happening is that to evaluate the face fluxes, for the:

- Convective term $\rightarrow$ will need velocity values at face (i.e. U_f)
- Diffusive term $\rightarrow$ will need velocity gradient at face (i.e. ∇U_f)

Now how do we get those face values. Note that we only have cell center values so we will have to use those in order to determine the face value.

For the velocity values at the face, you can use an interpolation whether that be linear, rhie and chow, etc. For the velocity gradient at the face, you can take the gradient using the cell centers adjacent to the face. Now what would that look like for the velocity gradient at the face? Consider the same diagram again but with some new labels and also consider Equation (2) which shows that that face velocity gradient would be.



$$\nabla U_f = \frac{U_N - U_P}{|\vec{d}_{PN}|} \quad (2)$$

(gives us a scalar face gradient in the direction of $\vec{d}_{PN}$)

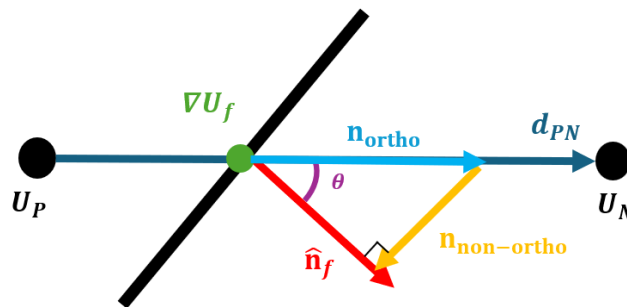
The problem and why non-orthogonality can be seen here! The issue is that the gradient we are computing (∇U_f) is in the direction of $\vec{d}_{PN}$. In a finite volume formulation using the divergence theorem (which, again, I won't be explaining in detail), what we really want is the gradient in the direction of $\hat{n}_f$. So, not only are we off... we are way off for the face value of the gradient.

We went through this diagnosis of the problem for the diffusive momentum flux but note that this problem is going to occur for *any diffusive flux* since those will require the gradient value at the face. How are we going to resolve this error in the computation of face gradients that show up in our diffusive fluxes...

The Solution

General Idea

The solution is actually kind of simple. In fact, you can probably think of it on your own. The basic idea is to decompose the gradient that we want (gradient in the direction of $\hat{n}_f$) into an orthogonal component (gradient in the direction of $\overrightarrow{d_{PN}}$) and a non-orthogonal component (which would point from $\overrightarrow{d_{PN}}$ to $\hat{n}_f$). This is shown in the diagram below.



(over-relaxed approach decomposition of the face normal vector)

A subtle point that can get lost here is that there are multiple different ways we can decompose the face normal vector. The one shown in the diagram above is the over-relaxed approach but there is also the minimum correction approach and rotation approach which is detailed in this [video](#) (which was also linked on Page 1).

The shown over-relaxed approach sets the non-orthogonal vector and the face normal vector perpendicular to each other, but we could have easily made the orthogonal and non-orthogonal vectors perpendicular while keeping the same face normal vector (which is fixed anyway).

Regardless of how you decompose the normal vector, there must be an orthogonal component that points in the direction of $\overrightarrow{d_{PN}}$. The length of the orthogonal component dictates how much of the solution is determined by that implicit derivative and the non-orthogonal component dictates how much is determined by the correction. So there is a nuance in the selection of the decomposition in their directions and length. It is also important to note that though the face normal vector is of unit length, the non-orthogonal and orthogonal component do not have to be.

I kind of rambled here but the [video](#) which I cannot recommend enough (the guy is the g.o.a.t.) discusses the nuances and differences between the methods and why the over-relaxed approach is the preferred method today.

So mathematically, we can express our face gradient now as in Equation (3) below.

$$\nabla U_f = \left[\frac{U_N - U_P}{|\vec{d}_{PN}|} \cdot n_{ortho} \right] + \left[\frac{U_N - U_P}{|\vec{d}_{PN}|} \cdot n_{non-ortho} \right] \quad (3)$$

where [the first bracketed term] $\rightarrow$ orthogonal contribution

[the second bracketed term] $\rightarrow$ non-orthogonal contribution

The important thing to note here is that $\frac{U_N - U_P}{|\vec{d}_{PN}|}$ is a *scalar* and is a two-point gradient scheme built only on U_N and U_P . For the orthogonal component, this is fine because the vector between U_N and U_P is in the direction the orthogonal normal vector. However, this is not actually the face gradient. The non-orthogonal direction is not aligned with $\vec{d}_{PN}$ so the $U_N - U_P$ does not contain the tangential gradient information that the non-orthogonal correction is supposed to provide.

To be frank, it would be cheating. It would be kind of like upwinding when its not even needed. It is literally determining a change in all directions from just the change in one direction. If it is like isotropic or we actually want the change in that one direction, that's fine. But when we are applying it in another direction... it becomes a pretty bad approximation. What we should do is determine the change by using more information from our neighbors. Also note that the finite difference is not actually the gradient... it is an approximation. How good is that approximation is a question that should be asked?

Therefore, rather, what we do for non-orthogonal correction is to use a more accurate gradient for the face that is built on information of a wider stencil (as opposed to just the difference between the cell centers). This can be expressed as Equation (4) below.

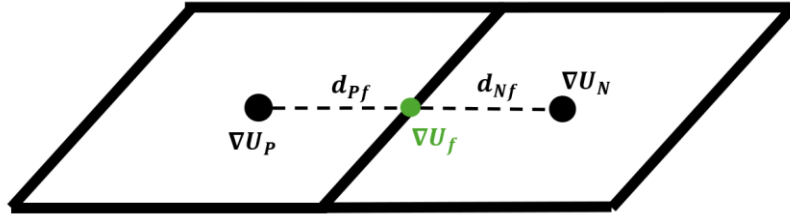
$$\nabla U_f = \left[\frac{U_N - U_P}{|\vec{d}_{PN}|} \cdot n_{ortho} \right] + [\overline{\nabla U_f} \cdot n_{non-ortho}] \quad (4)$$

where $\overline{\nabla U_f} \rightarrow$ face gradient determined from linear interpolation of cell center gradients

The $\overline{\nabla U_f}$ is a gradient at the face determined from cell center gradients. Where the cell center gradients are built from neighboring information i.e. these cell gradients are more accurate and use more information than the two-point gradient scheme.

Importantly though, notice that in Equation (3) the correction was implicit but now the correction will be explicit because we need the solution to evaluate this improved face gradient. So, the correction will kind of lag in the sense that it will be using the previous solution, but that is still better than using an implicit two-point gradient at the face.

The expression for $\overline{\nabla U_f}$ is shown in Equation (5) below using the corresponding notation from the diagram shown.



$$\overline{\nabla U_f} = \frac{d_{Pf} \nabla U_N + d_{Nf} \nabla U_P}{d_{Pf} + d_{Nf}} \quad (5)$$

As can be seen this is just a linear interpolation of the cell-center gradients. Simple enough. But the question is now how we get the cell-center gradients.

Green-Gauss Node-Based Gradient

This is where the idea of a Green-Gauss gradient is used. It is a method that can be used to determine the gradient which is as detailed in the following equations.

We start with the divergence theorem of the gradient of an arbitrary scalar phi (ϕ) as in Equation (6) then work our way to Equation (9) which is the Green-Gauss gradient. Note that area instead of volume is being used since the code is in 2D.

$$\int_A \nabla \phi \, dA = \int_A \nabla \cdot (\phi \mathbf{I}) \, dA = \oint_{\partial A} \phi \mathbf{I} \cdot \hat{n} \, dS = \sum_{faces} \phi_f \hat{n}_f S_f \quad (6)$$

$$\int_A \nabla \phi \, dA = A \nabla \phi \quad (7)$$

$$A \nabla \phi = \sum_{faces} \phi_f \hat{n}_f S_f \quad (8)$$

$$\nabla \phi = \frac{1}{A} \sum_{faces} \phi_f \hat{n}_f S_f \quad (9)$$

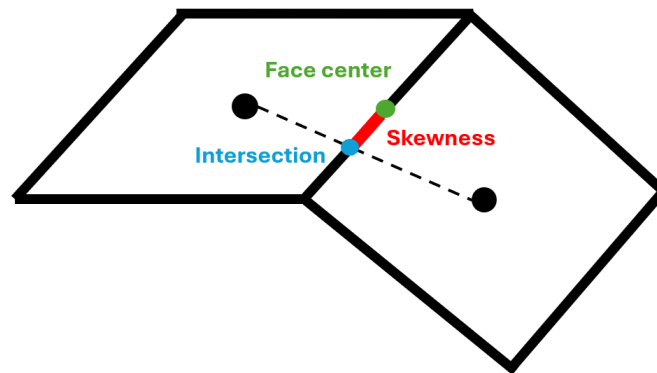
There is a big assumption here in assuming the gradient across the cell is constant which is what allows us to write Equation (7) but it is one we will take on for this approach. Also we used ϕ but for the diffusive momentum flux we have been considering $\phi \equiv U$.

So tracing back how we got here, we want the linearly interpolated cell-center gradient which requires us to get the cell-center gradient which is what we just did. Yet, to get the

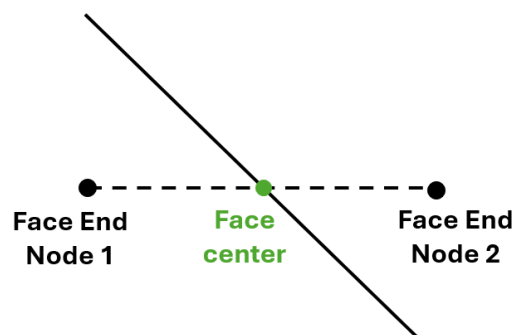
cell-center gradient using Equation (9) we need to find what ϕ_f is (or rather U_f for our diffusive flux case).

Previously we used linear interpolation, which is not bad, but we can do better. The issue with linear interpolation is really with skewness error. Previously, we talked about non-orthogonality but let us talk quickly about skewness now.

The skewness is another face metric which, in words, measures the deviation of the intersection of the vector between cell-centroids adjacent to the face and the cell center. An image is shown below since a picture is worth 1,000 words.



When we say we want the face center value when we decompose the divergence theorem into summation over faces, it's not just because "oooo let's try the middle". There's a rationale behind it. We are assuming a linear variation across the face. If the variation is linear, the value at the center must be the average across the entire face. A diagram is shown since this is another one of those subtle but important points.



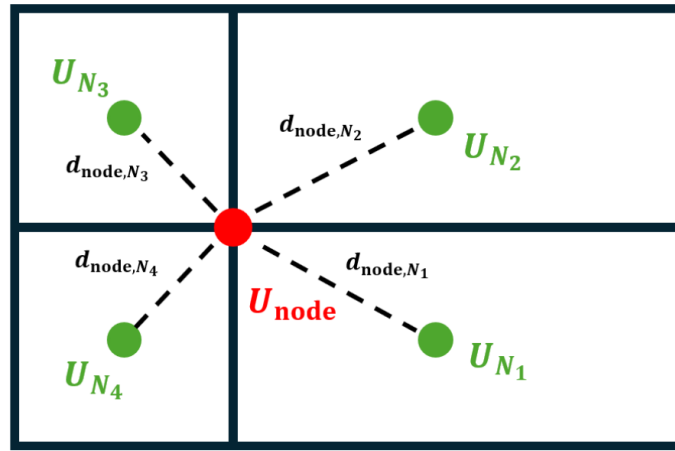
Maybe the idea is not getting conveyed well through this image (a bad 1,000 words I guess), but the idea is, again, a linear variation across a face means the average is the face center value so using the face center value is not just a "oooo let's try the middle".

So if linear interpolation results in skewness error, what can we do instead? We actually instead will compute the value at the face center using the values at the node instead of

the cell-center. And that is where the node-based part of the “node-based” gradient comes in. Whereas, the linear interpolation is more formally known as a “cell-based” gradient.

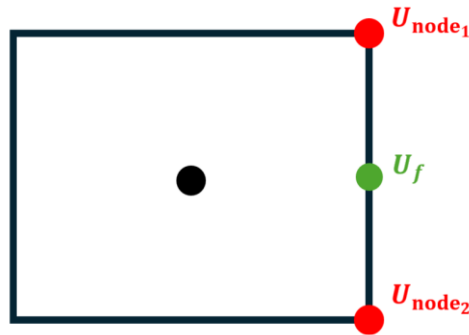
For the node-based gradient, we simply use a distance weighted average of cells neighboring a node. We would do that for the two nodes at the ends of our face then just take the average which will be that face center that we want (and the last part of this rabbit hole we went down to get the cell-center gradients).

The equation for the distance weighted average of cells neighboring a node is shown in Equation (10) below. Pretty straightforward hopefully especially given the beautiful diagram that comes along with it.



$$U_{\text{node}} = \frac{\sum_n^{\text{neighbors}} \left(\frac{1}{d_{\text{node},N_n}} U_{N_n} \right)}{\sum_n^{\text{neighbors}} \left(\frac{1}{d_{\text{node},N_n}} \right)} \quad (10)$$

Then Equation (11) below with the diagram is just to show the averaging that is done to get the value at the face center (which is the last piece!).



$$U_f = \frac{U_{\text{node}_1} + U_{\text{node}_2}}{2} \quad (11)$$

So let us just lay out these steps to be thorough since a lot was done.

1. We want a non-orthogonal correction for the face gradient when computing the diffusive flux

$$\nabla U_f = \left[\frac{U_N - U_P}{|\vec{d}_{PN}|} \cdot n_{ortho} \right] + [\overline{\nabla U_f} \cdot n_{non-ortho}] \quad (4)$$

2. We evaluate $\overline{\nabla U_f}$ using a linear interpolation of cell-centered gradients

$$\overline{\nabla U_f} = \frac{d_{Pf} \nabla U_N + d_{Nf} \nabla U_P}{d_{Pf} + d_{Nf}} \quad (5)$$

3. Compute the cell-centered gradients using a Green-Gauss gradient

$$\nabla U_P = \frac{1}{A_P} \sum_{faces} U_f \hat{n}_f S_f \quad (9)$$

4. For the face values of velocity (U_f) we use a node-based approach to avoid skewness error which first requires getting node values

$$U_{node} = \frac{\sum_n^{neighbors} \left(\frac{1}{d_{node, N_n}} U_{N_n} \right)}{\sum_n^{neighbors} \left(\frac{1}{d_{node, N_n}} \right)} \quad (10)$$

5. Then averaging those node values to get the face center value

$$U_f = \frac{U_{node_1} + U_{node_2}}{2} \quad (11)$$

Notice that these are just the steps for a single face. So there was a lot that needs to get implemented but so long as we follow these steps we should be good. So this is the heart of non-orthogonal corrections using a node-based Green-Gauss gradient.

Honestly very simple conceptually, let's hope it's also simple for ...

The Implementation

Where to Implement this Correction

So yes we have this correction and know how to do it but where do we actually implement it? As aforementioned, the non-orthogonal error comes from computing the gradients at faces which comes from fluxes of diffusive terms.

So, simple enough, what diffusive terms do we have in our SIMPLE implementation? There are two places:

1. The diffusive term in the momentum equation

$$(\mathbf{U} \cdot \nabla) \mathbf{U} - \nabla \cdot (\nu \nabla \mathbf{U}) = -\nabla p \quad (11)$$

(Equation (2) of the main document)

2. The diffusion of the pressure correction in the pressure Poisson equation (of the pressure correction step of the algorithm)

$$\nabla \cdot (\mathcal{A}^{-1} \nabla p') = \nabla \cdot \mathbf{U}^* \quad (12)$$

(Equation (12) of the main document)

You can see that in Equation (11) we will need the face value of the gradient $\nabla \mathbf{U}$, and for Equation (12) we will need the face value of the gradient $\nabla p'$.

So that is pretty cut and dry... those are where diffusive fluxes exist and hence where our corrections must be implemented.

How to Implement this Correction

I wrote this header before actually thinking about what would go in it. I actually am not really going to explain this because it is just coding. There is no intricacy in implementation details. It is just “for” loops, matrix multiplications, or however you want to code up Steps 1-5 on the previous page.

Note that in order to get the values at the nodes, you are going to have to also store information from the mesh that gives the neighboring cells of each node.

Actually, there is a nuance that is going to be in *iterative corrections*. What I mean by that is recall that the orthogonal component was implicit and the non-orthogonal component was explicit. Since it is explicit, it “lags”, so for very non-orthogonal cases, we will need multiple corrections. Specifically in the pressure correction equation. Consider this sequence of iterative refinement that is often done:

Iter 1 → Calculate $p'_{(0)}$ without any non-orthogonal correction

Iter 2 → Calculate $p'_{(1)}$ with non-orthogonal correction based on gradients using $\nabla p'_{(0)}$

Iter 3 → Calculate $p'_{(2)}$ with non-orthogonal correction based on gradients using $\nabla p'_{(1)}$

Iter 4 → ...

Notice that the non-orthogonal correction gets better each step since we update it based on the previous iteration's solution. More specifically, the gradient of the pressure correction $\nabla p'$ gets better over the iterations. For very non-orthogonal meshes, 2-3 corrections are the standard. For slightly non-orthogonal meshes, only 1 or none is needed.

A point to convey for understanding is that the mesh doesn't get any more non-orthogonal. Rather, the explicit correction term becomes a better approximation of the true non-orthogonal contribution as the lagged gradient is updated. The idea that I had to wrap my head around is circular dependence.

We essentially have $Ap' + C(p') = b$ type relationship where Ap' would be the implicit orthogonal term and $C(p')$ would be the explicit non-orthogonal term. Notice that we cannot evaluate the non-orthogonal term without p' but we are also solving for p' . So we use a guess for p' in $C(p')$ to solve for p' in Ap' .

Now, I kind of didn't want to do this, but this is a good refresher... why does it converge? Consider the growth/decay of the error term.

If the true solution is p'^* :

$$e^{(k)} = p'^* - p'^{(k)} \quad (13)$$

Now consider the current and true solution:

$$\text{current solution} \rightarrow p'^{(k+1)} = A^{-1} \left(b - C \left(p'^{(k)} \right) \right) \quad (14)$$

$$\text{true solution} \rightarrow p'^* = A^{-1} \left(b - C(p'^*) \right) \quad (15)$$

Take the difference between the true and current solution:

$$p'^* - p'^{(k+1)} = A^{-1} [b - C(p'^*)] - A^{-1} [b - C(p'^{(k)})] \quad (16)$$

Which becomes:

$$e^{(k+1)} = A^{-1} [C(p'^{(k)}) - C(p'^*)] \quad (17)$$

If C is linear (which we will assume for now) we can say $C(U) = CU$ then:

$$e^{(k+1)} = -A^{-1}C(e^{(k)}) \quad (18)$$

So, the growth/decay of the error term is determined by the spectral radius of $A^{-1}C$. Therefore, for convergence:

$$\rho(-A^{-1}C) < 1$$

Note that A is the implicit orthogonal diffusion coefficient and C is the explicit non-orthogonal correction coefficient. So as long as the implicit term is weighted larger than the explicit term, it should converge. The ratio of C/A is a measure of how non-orthogonal the mesh is where the more skewed a mesh, the smaller that ratio is meaning that we converge slower (i.e. spectral radius is smaller). This is why we need more iterations for a more non-orthogonal mesh.

I am not going to go into it but note that if C is non-orthogonal, the fixed point argument would still work but the convergence condition with the spectral radius will require a Jacobian at the solution. But let's not go into that for now...

So that's the idea of why we use iterative refinement for the pressure correction equation to better converge to the correct p' for highly non-orthogonal meshes.

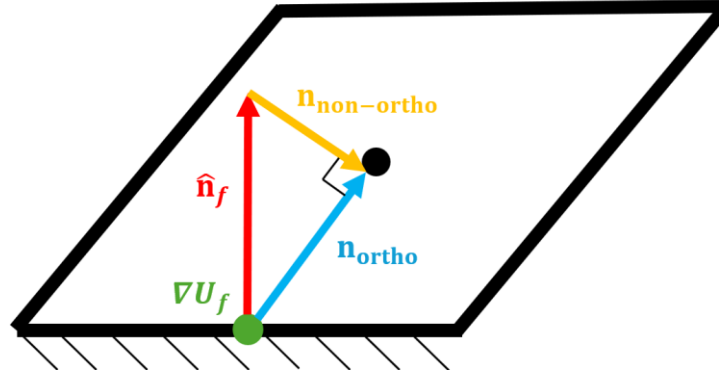
Now the last question is why do we not also do this same iterative refinement on the diffusive flux for the momentum equation. The answer to that is that the momentum equation is a transport equation and the pressure correction equation is a constraint equation. If the momentum equation is inaccurate, the pressure correction will fix it. If the pressure correction is inaccurate continuity is violated. The whole solution will deviate from the true solution.

The steps in the solver are really that we solve an incorrect momentum that the pressure correction is supposed to correct. So, the momentum equation can afford inaccuracy because the pressure correction will fix it but nothing is going to correct the mass imbalance from inaccurately solving the pressure correction. The iteration for the momentum is really in that outer loop of the simple whereas iteration for the pressure correction happens within the inner loops to satisfy continuity in one outer loop (that's a mouthful but hopefully that makes sense; and that's how I understand it but maybe there are other reasons as well).

So yes besides just implementing the equations as I showed them and updating the information you gather from the mesh, the iterative pressure correction solve is the only other nuance in the implementation.

Implementation at Boundaries

Lastly, as always, is the boundaries. We, as always, need to consider the boundaries. None of the ideas change. The only thing is that we now don't have a neighbor cell. We instead have the value at the face. Let me show a diagram just to make things more clear.



Different picture but this is no different to what was done before. The trick, so to say, is to use the face value U_f as the neighbor. Eh. Let me just reshoot the steps but this time for a boundary face. (Honestly, I am not sure if this is the standard way... I just kind of used my brain to think of what I'd do... maybe there's a more formal and better way... idk).

1. We want a non-orthogonal correction for the face gradient when computing the diffusive flux **this time use U_f instead of U_P**

$$\nabla U_f = \left[\frac{U_f - U_P}{|\vec{d}_{Pf}|} \cdot n_{ortho} \right] + [\overline{\nabla U_f} \cdot n_{non-ortho}] \quad (4)$$

2. We evaluate $\overline{\nabla U_f}$ using a linear interpolation of cell-centered gradients **but we don't know the cell-centered gradient at the boundary face so just use the next best thing: ∇U_P for ∇U_N**

$$\overline{\nabla U_f} = \frac{d_{Pf} \nabla U_P + d_{Nf} \nabla U_P}{d_{Pf} + d_{Nf}} = \nabla U_P \quad (5)$$

3. Compute the cell-centered gradients using a Green-Gauss gradient **this stays the same**

$$\nabla U_P = \frac{1}{A_P} \sum_{faces} U_f \hat{n}_f S_f \quad (9)$$

4. For the face values of velocity (U_f) we use a node-based approach to avoid skewness error which first requires getting node values

$$U_{\text{node}} = \frac{\sum_n^{\text{neighbors}} \left(\frac{1}{d_{\text{node}, N_n}} U_{N_n} \right)}{\sum_n^{\text{neighbors}} \left(\frac{1}{d_{\text{node}, N_n}} \right)} \quad (10)$$

5. Then averaging those node values to get the face center value

$$U_f = \frac{U_{\text{node}_1} + U_{\text{node}_2}}{2} \quad (11)$$

6. **However, note that at the boundary, we don't need this nodal value average to get the face center value. The face center value is prescribed by the boundary! (no approximation work needed)**

So yes it is the same steps with a couple caveats but nothing I think you should be scared of! You've got it!

Okay well that's the end of this document. We went through the problem, the solution, and the implementation. I don't really do conclusions, mostly because I am lazy. So until next time, bye.